

Day 3: Convolutional Neural Networks

Spatial Inductive Bias, Architectures, and the LeNet-5 Foundation

1. Why Fully-Connected Nets Fail on Images

Until now, we treated neural networks as stacks of Multi-Layer Perceptrons (MLPs), also known as fully-connected layers. While powerful for structured 1D data, MLPs catastrophically fail when applied to raw, high-dimensional images.

Parameter Explosion

Imagine processing a modest 1 Megapixel colored image ($1000 \times 1000 \times 3$). This translates to 3,000,000 input neurons. If our first hidden layer has just 1,000 neurons, the weight matrix connecting the input to this layer will have $3,000,000 \times 1,000 = 3 \text{ Billion}$ parameters.

Real-Life Example: A network with 3 billion parameters in just its first layer would require roughly 12 GB of RAM merely to store the weights (at 32-bit floats), completely overloading standard memory architectures, let alone the VRAM required for backpropagation gradients and batch processing. Furthermore, such massive capacity instantly leads to severe overfitting.

No Spatial Inductive Bias

An MLP flattens a 2D image grid into a 1D vector. By doing so, it utterly destroys the spatial topology of the image. The network does not know that pixel $(10, 10)$ is physically adjacent to pixel $(10, 11)$. It treats them with the exact same spatial independence as pixel $(10, 10)$ and pixel $(900, 900)$.

The Fix: CNNs introduce *Spatial Inductive Bias*. They assume that local neighborhoods of pixels are highly correlated (forming edges, textures) and that a feature useful in the top-left corner is equally useful in the bottom-right corner (Translation Invariance).

2. Convolution in a Neural Network

To solve the problems of MLPs, we use the mathematical operation of convolution, transitioning from static dense matrices to **learnable kernels**.

Learnable Kernels & Feature Maps

Instead of connecting every input pixel to a hidden neuron, we define a small window (e.g., 3×3 or 5×5) called a **kernel** (or filter). This kernel physically slides across the image. At each location, it computes a dot product between its weights and the underlying image patch.

Unlike classical image processing where we hardcode the kernel (like the Sobel edge detector), a CNN *learns* these weights via backpropagation.

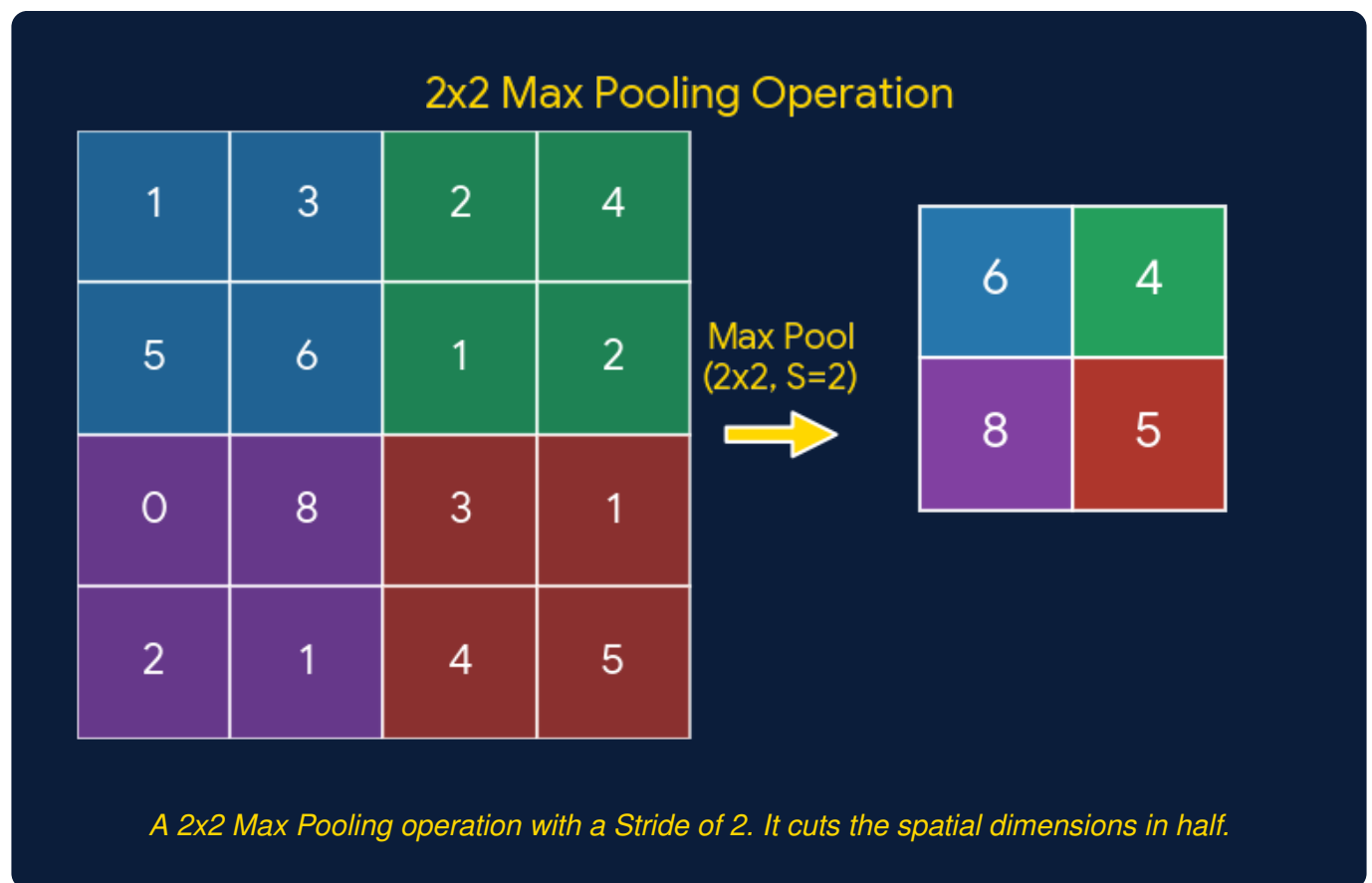
Multiple Filters: A single kernel extracts a single type of feature (e.g., vertical edges). To understand a complex image, we need multiple feature types. A convolutional layer applies N different kernels simultaneously. The output is a stack of N 2D matrices called **Feature Maps** (or channels).

Real-Life Intuition

Imagine you are looking at a large painting through a small magnifying glass (the kernel). You scan the painting left-to-right, top-to-bottom. If your brain is wired to look for the color yellow (your learned weight), it "fires" a high signal every time yellow appears in the glass. The resulting mental map of "where the yellow is" constitutes one Feature Map.

3. Pooling: Downsampling the Spatial Dimensions

Once a feature has been detected, its exact absolute location becomes less important; only its approximate relative position matters. Pooling layers aggressively downsample the spatial dimensions (**Height** $\times$ **Width**), drastically reducing the computational load and providing translational invariance.



Types of Pooling

- **Max Pooling:** Slides a window and takes the maximum value. *Why?* It extracts the most prominent, strongest feature in that neighborhood. It is the dominant choice in modern architectures (like VGG, ResNet).
- **Average Pooling:** Takes the arithmetic mean. *Why?* It smooths the features, retaining more generalized information, but dilutes strong activations. Used historically in LeNet-5.

- **Global Average Pooling (GAP):** A radical operation used at the very end of modern networks. It takes an entire feature map (e.g., 7×7) and averages all 49 values into a single number. It obliterates spatial dimensions completely, elegantly preparing the data for the final classification without needing massive Flatten/Dense layers.

4. Stride and Padding

Two hyperparameters strictly govern the geometric movement of kernels across the input matrix.

Stride (S)

Stride dictates the step size the kernel takes as it slides. A Stride of 1 means the kernel moves one pixel at a time. A Stride of 2 means it jumps two pixels, effectively skipping data and halving the output dimension directly within the convolution.

Padding (P)

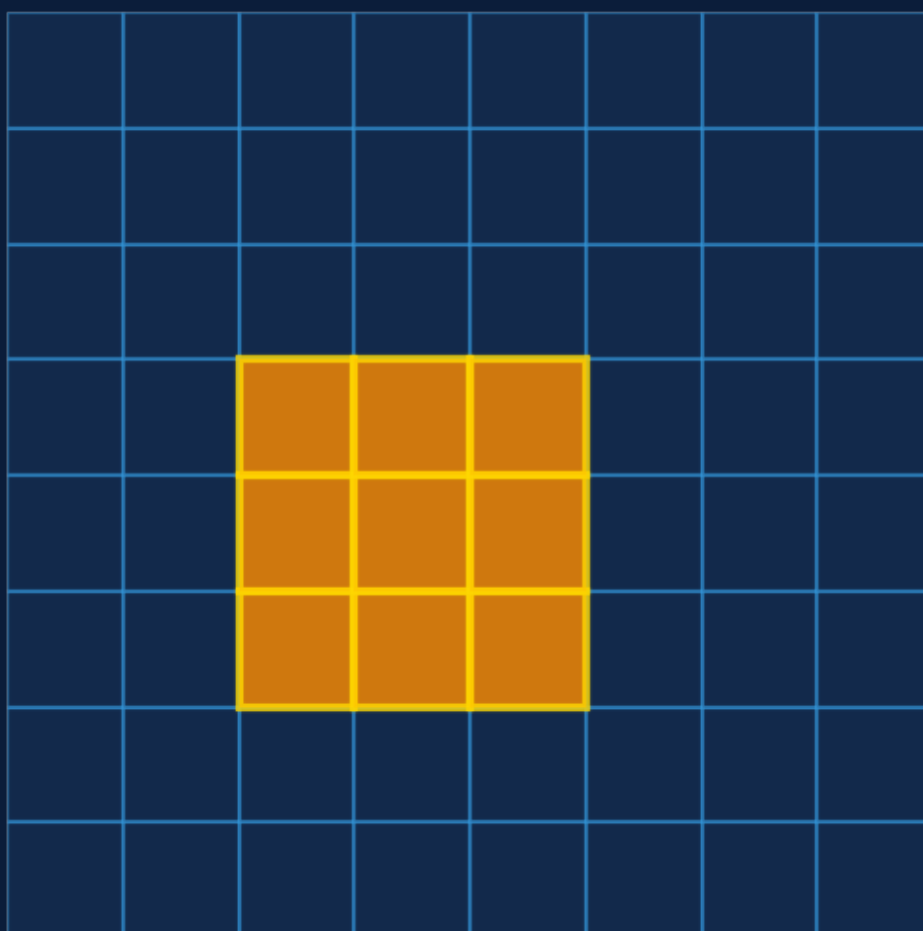
When a kernel slides over an image, it cannot center itself on the border pixels without falling off the edge. Without padding, the output feature map continuously shrinks after every layer. Padding involves wrapping the image in a border of zeroes (Zero-Padding).

Padding Type	Mechanism	Effect
'Valid' (No Padding)	Kernel strictly stays within the image boundaries.	Output dimensions shrink. Edge pixels are under-sampled.
'Same' (Zero Padding)	Pad with zeroes so the output size equals the input size (assuming Stride=1).	Preserves spatial dimensions. Formula for 3 × 3 kernel: Pad by 1. For 5 × 5 : Pad by 2.

5. Receptive Field

The Receptive Field (RF) defines the region of the original input image that influences a specific neuron in the network. Understanding RF is critical for debugging why a network fails to recognize large objects.

3x3 Receptive Field on an 8x8 Input



A neuron in the feature map observes only a localized patch of the input grid.

The Accumulation Effect: As you go deeper into the network, the receptive field grows linearly with convolutions and exponentially with pooling.

- Layer 1 (Conv 3x3): A neuron sees a **3 × 3** pixel patch of the input.

- Layer 2 (Max Pool 2x2): Downsamples by 2.
- Layer 3 (Conv 3x3): A neuron here sees a 3×3 patch of Layer 2. But because Layer 2 was pooled, this neuron actually sees a roughly 10×10 patch of the original input image!

Real-Life Context: If you are building a CNN to detect a cat that occupies 150×150 pixels in an image, but the receptive field of your deepest layer is only 40×40 pixels, the network will physically never be able to "see" the whole cat at once, leading to catastrophic failure. You must add more pooling layers or larger kernels to expand the RF.

6. The Standard CNN Block Pattern

Through decades of empirical research, a standard micro-architecture has emerged as the bedrock of modern vision models (ResNet, EfficientNet).

Conv → BatchNorm → ReLU → Pool

- **1. Convolution:** Extracts the affine features. Maps input channels to output channels.
- **2. Batch Normalization (BN):** Normalizes the output tensor to have a mean of 0 and variance of 1. It stabilizes the gradients, prevents the "Internal Covariate Shift," and acts as a mild regularizer, allowing much higher learning rates.
- **3. ReLU:** The non-linear activation function. Placed *after* BN because normalizing pre-activation distributions is mathematically superior.
- **4. Pooling:** Downsamples the spatial resolution. (Note: Modern architectures sometimes omit pooling in favor of Convolutions with Stride=2).

7. The Golden Output Size Formula

You must memorize this formula to calculate the exact shape of tensors flowing through PyTorch layers. If you mismatch shapes, your matrix multiplications will crash.

$$\text{Output Size} = \text{floor}((H + 2P - K) / S) + 1$$

Where:

- **H**: Input dimension (Height or Width).
- **P**: Padding applied to one side.
- **K**: Kernel size.
- **S**: Stride.

Calculation Example

Assume an input image of **224 × 224**. We apply a Conv layer with a **7 × 7** Kernel, Stride of 2, and Padding of 3.

```
H = 224, P = 3, K = 7, S = 2
Numerator = 224 + (2 * 3) - 7 = 224 + 6 - 7 = 223
Division = 223 / 2 = 111.5
Floor + 1 = floor(111.5) + 1 = 111 + 1 = 112
Output Shape: 112 × 112
```

8. LeNet-5: The Foundational Architecture

Based on the seminal 1998 paper by Yann LeCun et al., *Gradient-Based Learning Applied to Document Recognition*, LeNet-5 is the grandfather of all modern CNNs. It was designed to read handwritten digits (0-9) on bank checks using the MNIST dataset.

The Network Topology (Sections 1 & 2)

The architecture rigidly follows a **Conv-Pool-Conv-Pool-FC-FC-FC** sequence. Note the historical differences: it uses Tanh activations and Average Pooling, which we will modernize in our PyTorch task to ReLU and Max Pooling.

Layer	Type	Kernel (K)	Stride (S)	Padding (P)	Input Shape	Output Shape	Trainable Params
Input	Image	-	-	-	$1 \times 32 \times 32$	$1 \times 32 \times 32$	0
C1	Convolution	5×5	1	0	$1 \times 32 \times 32$	$6 \times 28 \times 28$	156
S2	Average Pool	2×2	2	0	$6 \times 28 \times 28$	$6 \times 14 \times 14$	12 (Weights/ Biases)
C3	Convolution	5×5	1	0	$6 \times 14 \times 14$	$16 \times 10 \times 10$	1,516
S4	Average Pool	2×2	2	0	$16 \times 10 \times 10$	$16 \times 5 \times 5$	32
C5	Convolution (FC)	5×5	1	0	$16 \times 5 \times 5$	$120 \times 1 \times 1$	48,120
F6	Fully Connected	-	-	-	120	84	10,164
Output	RBF (Logits)	-	-	-	84	10 (Classes)	840

9. PyTorch Task Theory: Modernizing LeNet-5

Your task requires building LeNet-5 exactly as dimensionally described, but upgrading the mechanics to modern standards (Day 2 mappings). Here is the theoretical framework required to complete the exercise.

1. Architecture Equivalence

- **Modern Substitutions:** The original paper used ``Tanh`` and ``AvgPool2d`` with learnable scaling. Modern standard replaces this with ``nn.ReLU`` and standard ``nn.MaxPool2d``. The spatial dimensions remain identical, but gradients flow much faster preventing vanishing gradients.
- **C5 as a Linear Layer:** While C5 is technically a Convolution ($16 \times 5 \times 5$ input to 120 channels using a 5×5 kernel), it mathematically collapses into a dense layer because the kernel size perfectly matches the input size (5×5). In PyTorch, you will use `nn.Flatten()` followed by `nn.Linear(16 * 5 * 5, 120)`.
- **F6 and Output:** F6 is an 84-neuron dense layer. Why 84? The original paper used 7×12 ASCII bitmap representations. You will implement this simply as `nn.Linear(120, 84)`. The final output is `nn.Linear(84, 10)`.

2. Training Loop Architecture

A rigorous training loop requires managing state correctly across batches:

- **DataLoader:** You must wrap your MNIST tensors in a `DataLoader` to handle batching and shuffling (SGD requirement).
- **Adam Optimizer:** Unlike LeCun's diagonal Levenberg-Marquardt approximation (Appendix C of the paper), Adam handles adaptive momentum per parameter automatically, converging drastically faster.
- **StepLR:** Learning Rate decay is crucial. As you descend into the loss basin, you must reduce step size. StepLR multiplies the LR by a factor (gamma) every N epochs.

10. Visualizing Kernels & Metrics Tracking

Visualizing the Conv1 Filters

Your task requires visualizing the 32 learned filters in the first conv layer (assuming you expand C1 from 6 to 32 for a modern variant, or visualize a specific setup). Theoretical backing:

- The weights of a PyTorch Conv2d layer are stored in `layer.weight.data`.
- For the first layer with 1 input channel and 32 output channels, the weight tensor shape is `[32, 1, 5, 5]`.
- These are literally just 32 separate **5 × 5** images. Using Matplotlib subplots, you can plot them in an **8 × 4** grid. They will visually represent the Gabor-like edge detectors the network learned.

Comparison: MLP vs CNN

When compiling your report, expect the following theoretical outcomes:

- **Parameters:** The MLP will have significantly more parameters than LeNet-5, despite LeNet-5 being a "deeper" network. This is the power of *Weight Sharing* in convolutions.
- **Accuracy:** LeNet-5 will obliterate the MLP (expect ~98-99% vs ~95%). The inductive bias of local spatial neighborhoods allows LeNet to recognize a "loop" of an '8' regardless of whether it shifted two pixels to the left.
- **Training Time:** LeNet-5 may take slightly longer per epoch due to the sliding window mathematics, but it will converge in fewer epochs.